mc6801-tools 2.0: Uninitialized Data and a Tighter Layout

mc6801-tools

November 2026

Contents

1 Summary **2**

2 Background: how it works today **3**

2.1 Location counters 3

2.2 Common blocks 3

2.3 Padding 3

2.4 NOLOAD 3

3 Goals and non-goals **3**

3.1 Goals 3

3.2 Non-goals 4

4 .BSS gets its own counter **4**

5 The default layout **4**

6 NOLOAD and initialized data **5**

7 Warnings on by default **5**

8 Clearing uninitialized data **5**

9 Compatibility and migration **6**

10 Implementation **7**

10.1 1. ld01: initialized data in NOLOAD sections (1.x) 7

10.2 2. as01: the .BSS counter 7

10.3 3. ld01: the default layout 7

10.4 4. Documentation 7

11 Testing **7**

12 Phases **8**

13 Decisions **8**

1 Summary

Two things in the default layout waste space in a program. They can't be changed without changing where things end up, so they are proposed together for a major version, with a third change that a major version is the time for:

1. **Uninitialized data takes space in the file.** `as01` puts `.BSS` in the same location counter as `.DATA`, so `RMB` space reserved under `.BSS` is written to the executable, and loaded from cassette or stored in ROM, as zeros. Only common blocks after the last data are uninitialized today.
2. **Each module's part of each section is padded to a multiple of 4 bytes**, and so is each common block. The MC6801 has no alignment requirements, so this is waste inherited from the 8086 tools that `ld01` comes from: the demo program's 108 bytes of code become 99 without it.

The proposal:

- `.BSS` (and `.SECT .bss`) selects **location counter 4**, and the default layout puts counter 4 and all common blocks in a **`NOLOAD .bss` section** after `.data`: uninitialized data in the a.out header, not in the file.
- The default layout **drops the 4-byte padding**, and `ld01` **stops rounding common blocks** up to 4 bytes.
- `ld01` reports **initialized data in a `NOLOAD` section** as an error, instead of silently leaving it out. This can come first, in 1.x.
- `as01` **reports warnings by default**; `-w` turns them off.

The object format, the linker script language and the a.out format don't change. A program that doesn't use `.BSS` or common blocks, and is linked with a script of its own, links exactly as before.

2 Background: how it works today

2.1 Location counters

as01 has 16 location counters. .TEXT selects counter 0, and .DATA, .ROM and .BSS all select counter 3 (BSSLOC and DATALOC are both 3 in as/const.h). LOC n selects counter n. In linker scripts, .text means LOC0 and .data means LOC3.

RMB (and .BLKB) reserves space without giving it a value. In an object file it is a "skip" record, and ld01 writes the skipped bytes as zeros, like any other part of a loaded section. So space reserved under .BSS is in the file, as zeros, just as if it were .DATA.

2.2 Common blocks

A common block (COMM) is an input section of its own, COMMONn, for counter n. The default script puts each counter's common blocks right after its LOCn input in .data, and works out the end of the initialized data (__edata) as the end of the last counter with contents: only the common blocks after it are uninitialized data, left out of the file. ld01 rounds each common block up to a multiple of 4 bytes (ld/writex86.c).

2.3 Padding

The default script gives .text and .data SUBALIGN(4): each module's part of each counter starts at a multiple of 4 bytes, and the gaps are written as zeros. Scripts of the user's own don't do this unless they ask for it.

2.4 NOLOAD

A NOLOAD output section isn't written to the output; in a.out output, a NOLOAD section after the data becomes the uninitialized data (the bss.ld test). If a module puts initialized bytes, or relocations, in a counter that a script places in a NOLOAD section, ld01 leaves them out without a word.

3 Goals and non-goals

3.1 Goals

- Space reserved under .BSS, and every common block, takes no space in the executable, the cassette image or ROM.
- No padding between modules in the default layout.
- A mistake that would lose data, such as FCB under .BSS, is an error.
- A program can get the 1.x layout back, as far as possible, with a linker script.

3.2 Non-goals

- Clearing uninitialized data at run time. The tools can't do that for a program; start-up code does it, using symbols the linker provides (see Clearing uninitialized data).
- Changing the object format, the a.out format or the linker script language.

4 .BSS gets its own counter

`.BSS`, and `.SECT .bss`, select **location counter 4**. `.DATA` and `.ROM` still select counter 3. In linker scripts, `.bss` means `LOC4`, as `.data` means `LOC3`.

Counter 4 follows `.DATA`'s counter 3, and keeps the counters after it free for programs to use with `LOC n`. (Counter 2 is out: `ld01` keeps it within one 256-byte page.) The user's manual suggests `LOC 4` for page-zero data with `DPAGE`; that suggestion, and its examples, move to counter 5.

There is no option to keep `.BSS` in counter 3: a program that needs 1.x's placement writes `.DATA` instead.

`as01`'s `-t n` option, which puts `.TEXT` in counter n+3, keeps counting from 3; it uses `BSSLOC` today and must use `DATALOC` instead.

`as01` doesn't check what goes into counter 4: the linker script decides whether a counter is loaded, so the check belongs to `ld01` (see NOLOAD and initialized data).

5 The default layout

The default script (`ld01 -P`) becomes:

```
SECTIONS
{
    .text 0x0000 :
    {
        __btext = .;
        *(LOC0)
        *(LOC1)
        __etext = .;
    }

    .data __etext :
    {
        __bdata = .;
        ONEPAGE { *(LOC2) }
        *(LOC3) *(LOC5) *(LOC6) ... *(LOC15)
        __edata = .;
    }

    .bss (NOLOAD) :
```

```
    {
        *(LOC4)
        *(COMMON)
        __end = .;
    }

    PROVIDE(__heap_top = __end + 0x0000);
}
```

(The `PROVIDE(__segnXX)` symbols for each counter stay, as today; they are left out above.)

- **No `SUBALIGN(4)`**: modules follow each other with no gaps.
- **All common blocks** go in `.bss`, so `__edata` is simply the end of `.data`, and the `__dend` bookkeeping goes away.
- **`.bss` is `NOLOAD`**: in an a.out file, its size is the `a_bss` field, and it isn't in the file. `objcopy01` already converts only the text and data, so cassette images and S-records don't contain it.
- The `-T`, `-D`, `-H`, `-i` and `-d` options work as before.

`ld01` no longer rounds common blocks up: a 1-byte `COMM` takes 1 byte.

6 NOLOAD and initialized data

`ld01` reports initialized data in a counter that a script places in a `NOLOAD` section, naming the module and the counter:

```
ld01: main.o: LOC4 has initialized data, but .bss is NOLOAD
```

Space reserved with `RMB`, and common blocks, are fine. This check doesn't depend on the rest of the proposal: today, such data is silently lost, so it is a bug fix that can be released in 1.x first.

7 Warnings on by default

`as01` reports warnings only with `-w-` or `.WARN 1`, so mistakes it can see, such as a `JSR` that could be a `BSR`, go unnoticed. In 2.0, warnings are on by default: `-w` turns them off (and `-w-` is still accepted), and `.WARN` works as before. Warnings don't change the exit status or the output, so builds keep working; the demo and the manual's examples assemble without any.

8 Clearing uninitialized data

Uninitialized data isn't in the program as loaded, so it holds whatever was in memory: on the MC-10, whatever BASIC or an earlier program left there. This is already true of common blocks after the data today; in 2.0 it is true of everything under `.BSS` too.

A program that relies on that memory being zero must clear it in its start-up code. Every output section gets `__name_start` and `__name_end` symbols, so the default layout provides `__bss_start` and `__bss_end`:

```
        IMPORT  __bss_start, __bss_end
        ldx     #__bss_start    ; clear .BSS and the common blocks
clear   cpx     #__bss_end
        beq     cleared
        clr     0,x
        inx
        bra     clear
cleared
```

The demo's start-up module doesn't need this (it has no uninitialized data); the user's manual's examples will show it.

9 Compatibility and migration

Program	Effect in 2.0
Uses only `.TEXT`, `.DATA` and `LOC` 0–3, 5–15, default layout	Smaller: no padding. Addresses move.
Uses `.BSS` or `COMM`, default layout	Smaller file; uninitialized data must be cleared by the program if it relies on zeros
Uses `LOC 4` for initialized data, default layout	Link error (initialized data in `NOLOAD .bss`); move it to another counter
Uses `LOC 4` for page-zero data, as the 1.x manual suggests	Works with its own script, which places counter 4; the manual now uses counter 5, to keep counter 4 for `.BSS`
Linked with its own script	Unchanged, unless it uses `.BSS` (now counter 4, which a pattern for the data counters such as `*(LOC2 LOC3 LOC4 ...)` still takes) or common blocks (no longer rounded up to 4 bytes)
Any, assembled	Warnings now shown; `-w` gives the 1.x behavior

To get the 1.x default layout back, save the 1.x default script (`ld01 -P` from version 1.6) and link with it (`-S`): it has the padding and the old placement of common blocks. Two things can't be undone that way:

- `.BSS` is in counter 4, which `as01` decides. With the 1.x script, counter 4 is placed in `.data` after counter 3 and written as zeros, as before, but in a different place. Writing `.DATA` instead of `.BSS` restores the 1.x placement exactly.
- **Common blocks** are no longer rounded up to 4 bytes, which `ld01` decides whatever the script. Only a program that uses more of a common block than it declares can notice.

10 Implementation

10.1 1. ld01: initialized data in NOLOAD sections (1.x)

When a module's counter is placed in a `NOLOAD` section, `ld01` already marks it (`segnoload`). While writing the module, an absolute-data record or a relocation in such a counter is an error; a skip record isn't. Report it once per module and counter, with a new function in `ld/io.c` like `byte_reloc_error()`.

10.2 2. as01: the .BSS counter

- `BSSLOC` becomes 4 in `as/const.h`; `-t` uses `DATALOC`.
- `.SECT .bss` follows `.BSS`, as it already calls `pbss()`.
- Warnings are on by default (`as_warn`'s initial state), and `-w` turns them off.

10.3 3. ld01: the default layout

- `default_script()` in `ld/script.c`: no `SUBALIGN(4)`, the `.bss` section with `LOC4` and the common blocks, and no `__dend`.
- `.bss` in input patterns means `LOC4` (`secmatch()`).
- No rounding of common blocks (`ld/writex86.c`).

10.4 4. Documentation

- `as01(1)` and the manual: `.BSS` selects counter 4, and what that means; warnings on by default (`-w` to turn them off), and the "warnings are off by default" known limitation goes.
- The manual's page-zero examples (`zpvars.s`, `zpmain.s`, and the text around them) and `as01(1)`'s `DPAGE` example use counter 5.
- `ld01(1)` and the manual: the default layout, `.bss` in patterns, the new error, and clearing uninitialized data; the "common blocks are allocated in multiples of 4 bytes" note goes.
- A "Migrating to 2.0" section in the manual, and in the release notes.

11 Testing

Nearly every link with the default layout changes, so the tests can't just keep their reference outputs; the new ones are checked like this:

- **Running:** the demo and the ROM examples run on the simulator with the same results as in 1.x, and a new test program with `.BSS` data and common blocks, cleared by its start-up code, runs and checks the values it leaves.
- **Layout:** the `-M` maps show no gaps between modules; the a.out headers show `.BSS` and the common blocks in `a_bss`, and the files are that much smaller.
- **Round trip:** linking with `ld01 -P`'s output still gives the same result as linking without a script.

- **1.x layout:** for the programs that don't use `.BSS`, linking with the saved 1.6 default script gives byte-identical output to 1.6.
- **The new error:** `FCB` under `.BSS`, and initialized data in a `NOLOAD` section of a user script.
- **Warnings:** a source with a warning reports it without `-w` and not with it, and the exit status is the same either way.

12 Phases

Phase	Work	Release
1	`ld01`: initialized data in `NOLOAD` is an error	1.6.1 (bug fix), the last 1.x release
2	`as01`: `.BSS` in counter 4, warnings on by default; `ld01`: the new default layout, `.bss` in patterns, no common rounding	2.0.0
3	Documentation, examples (page-zero data in counter 5), migration notes	2.0.0

Phases 2 and 3 are released together, since 2 alone would change the layout without documenting it. There is no 1.x maintenance branch after 2.0: fixes go into 2.x only.

13 Decisions

These questions were settled when the design was reviewed:

1. **The `.BSS` counter** is 4, following `.DATA`'s counter 3. The manual's suggestion for page-zero data moves from counter 4 to 5.
2. **No option** to keep `.BSS` in counter 3: writing `.DATA` instead does the same.
3. **Common blocks** are no longer rounded up to 4 bytes.
4. **Warnings** are on by default in `as01`; `-w` turns them off.
5. **No 1.x maintenance branch**: phase 1 can be the last 1.x release (1.6.1), and fixes after 2.0 go into 2.x.